pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Exploratory Data Analysis

WEEK 3, DAY 3

Created by **Jeff Lenamon**

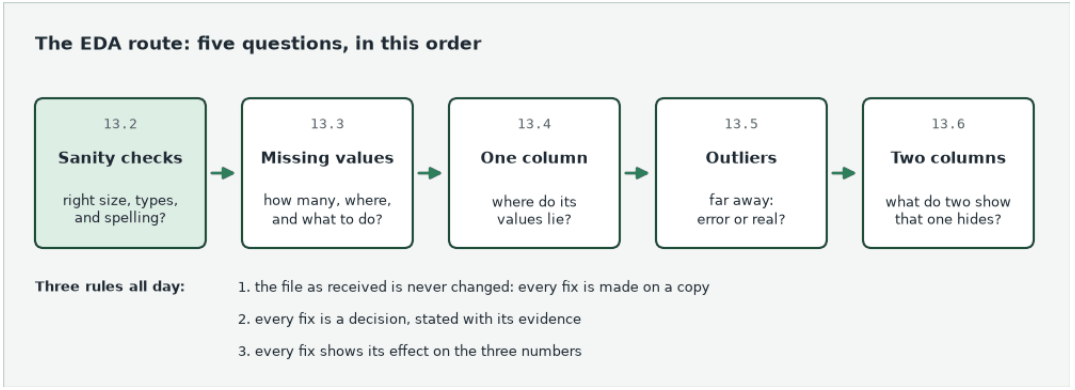
This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: can the file be trusted?

Ops sent the month-end extract, "holdings_messy.csv", as of 2026-09-30.

Their cover note says 200 bonds, 489,000,000 of face, 490,819,215 of market value. I need three numbers for the pack: market value, average OAS, average yield. Before any of them leaves the desk, tell me if the file can be trusted. Fix what you can prove, flag what you cannot, show me what each fix does to the three numbers, and give me the list for ops. Then make it run on next month's file in one line.

The issuers are invented; every statement describes the data. Getting to know a table before relying on it is exploratory data analysis, or EDA.



Three numbers, one file, and nothing leaves the desk until the file adds up.

13.1 The Extract and the Cover Note

```
raw = pd.read_csv(data_folder + 'holdings_messy.csv')
ratings = pd.read_csv(data_folder + 'ratings.csv')
print(raw.shape, ratings.shape)
(204, 19) (18, 4)
```

```
expected_bonds = 200
expected_face = 489000000
expected_value = 490819215
headline('As received', raw)
As received | rows: 204 | market value: 497769925.0 | OAS: 187.83 | yield: 6.051
```

204 rows against 200: a finding before the first check.

13.2 Size against the note, and dates held as text

```
print('Rows:', len(raw) - expected_bonds)
print('Face:', raw['par_held'].sum() - expected_face)
print('Market value:', raw['market_value'].sum() - expected_value)
Rows: 4
Face: 6500000
Market value: 6950710.0
```

All three are over, none under: something was added.

```
raw['price_date'].max() - raw['price_date'].min()
TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

"max()" ran, because text sorts; the minus failed: the dates arrived as text, as in Combining Part 2. Older pandas, as in Colab, calls a text column "object", not "str".

Every date read; always print that count, because a coerce turns a bad date into a silent gap.

```
book = raw.copy()
for column in ['as_of_date', 'maturity', 'price_date']:
    book[column] = pd.to_datetime(book[column], format='%Y-%m-%d', errors='coerce')
    print(column, 'not read:', book[column].isna().sum())
print('Prices from', book['price_date'].min(), 'to', book['price_date'].max())
print('As-of dates:', book['as_of_date'].nunique())
as_of_date not read: 0
maturity not read: 0
price_date not read: 0
Prices from 2026-08-04 00:00:00 to 2026-09-30 00:00:00
As-of dates: 1
```

One as-of date, yet the prices span 57 days: hold that thought.

```
import io
csv_text = 'cusip,price\n00000AA01,100.5\n00000AA02,missing\n00000AA03,98.25\n00000AA04,--\n'
prices = pd.read_csv(io.StringIO(csv_text))['price']
print(prices.dtype, '| not numbers:', pd.to_numeric(prices, errors='coerce').isna().sum())
print(pd.read_csv(io.StringIO(csv_text), na_values=['missing', '--'])['price'].dtype)
str | not numbers: 2
float64
```

"NA" and an empty cell are missing already; "na_values=" adds a system's own words.

Rows sent twice, and tickers spelled two ways

```
print('Rows:', book.duplicated().sum(), '| CUSIPs:', book['cusip'].duplicated().sum())
print('Rows marked with keep=False:', book.duplicated(keep=False).sum())
copies = book[book.duplicated()]
print('Copies at rows:', copies.index.tolist())
print('Face:', copies['par_held'].sum(), '| market value:', copies['market_value'].sum())
Rows: 4 | CUSIPs: 4
Rows marked with keep=False: 8
Copies at rows: [200, 201, 202, 203]
Face: 6500000 | market value: 6950710.0
```

Four rows and four CUSIPs, so no CUSIP comes back with other values. The copies sit at the end of the file and carry the cover note's two gaps to the dollar.

Decision 1: Remove the Copies

```
book = book.drop_duplicates().reset_index(drop=True)
headline('Decision 1', book)
print('Off the note:', len(book) - expected_bonds, book['par_held'].sum() - expected_face,
      book['market_value'].sum() - expected_value, '| raw still has', len(raw), 'rows')
Decision 1 | rows: 200 | market value: 490819215.0 | OAS: 185.75 | yield: 6.028
Off the note: 0 0 0.0 | raw still has 204 rows
```

Both totals tie to the dollar: the four copies were the whole gap.

```
print('CUSIP lengths:', book['cusip'].str.len().value_counts().to_dict())
print('Issuers:', book['issuer'].nunique(), '| tickers:', book['ticker'].nunique())
tidy = book['ticker'].str.strip().str.upper()
untidy = book['ticker'] != tidy
print('Untidy:', untidy.sum(), book.loc[untidy, 'ticker'].tolist())
CUSIP lengths: {9: 200}
Issuers: 115 | tickers: 121
Untidy: 8 ['cora', 'CORX ', 'dran', ' elvl ', ' FESH', ' thaa ', 'WYXR ', ' XANE']
```

All 200 CUSIPs have nine characters, but 115 issuers carry 121 tickers, and eight tickers are untidy.

Decision 2: Tidy the Tickers

```
print('THAA as received:', book.groupby('ticker')['market_value'].sum()['THAA'])
book['ticker'] = tidy
print('THAA tidied:', book.groupby('ticker')['market_value'].sum()['THAA'])
print('Tickers:', book['ticker'].nunique(), '| issuers:', book['issuer'].nunique())
headline('Decision 2', book)
THAA as received: 2659800.0
THAA tidied: 4517617.5
Tickers: 115 | issuers: 115
Decision 2 | rows: 200 | market value: 490819215.0 | OAS: 185.75 | yield: 6.028
```

As received, THAA was short 1,857,817.5, and nothing warned. The headline does not move: a fix can matter to one report and not to another.

The rating scale, and the August prices

```
untidy_sectors = book['sector'] != book['sector'].str.strip().str.title()
print('Sectors:', book['sector'].nunique(), '| untidy:', untidy_sectors.sum())
known = book['rating'].isin(ratings['rating'])
off_scale = book.loc[~known, 'rating'].value_counts(dropna=False)
print('On the scale:', known.sum(), '| off it:', off_scale.to_dict())
Sectors: 10 | untidy: 0
On the scale: 195 | off it: {nan: 5}
```

Five ratings are missing, not misspelled: 13.3 takes them up.

No bond breaks a rule, and market value rebuilds to the cent.

```
stale = book[book['price_date'] < book['as_of_date']]
days_old = (stale['as_of_date'] - stale['price_date']).dt.days
print(len(stale), 'bonds,', days_old.min(), 'to', days_old.max(), 'days old')
share = stale['market_value'].sum() / book['market_value'].sum() * 100
print('Market value:', stale['market_value'].sum(), '|', round(share, 2), 'percent')
6 bonds, 35 to 57 days old
Market value: 13342347.5 | 2.72 percent
```

Decision 3: flag the six for ops and change nothing. The right price is not in the file, and an invented price is worse than an old one.

Excel side by side. Price in I, price date in J, market value in S

Check	pandas	Excel
Rows and totals	`len(raw), .sum()`	=COUNTA(B2:B205), =SUM(S2:S205)
Number or text	`dtypes`	=ISNUMBER(I2), =ISTEXT(I2)
Words for missing	`na_values=`	Power Query, Replace Values with null
Rows sent twice	`duplicated()`	=COUNTIF(B:B, B2)>1; Remove Duplicates deletes in place
Untidy text	`.str.strip().str.upper()`	=UPPER(TRIM(C2))
On the scale	`isin()`	=COUNTIF(ratings!A:A, F2)>0; Data Validation, List
Stale prices	`price_date < as_of_date`	=COUNTIF(J:J, "<"&DATE(2026, 9, 30))

Excel's = ignores case, so ="thaa"="THAA" is TRUE; EXACT tells them apart.

Two fixes the file proves, one flag it cannot settle. Next, what is not there.

13.3 Five bonds with no rating, and a label for them

```
gaps = book.isna().sum()
print('Gaps:', gaps[gaps > 0].to_dict())
book = pd.merge(book, ratings, on='rating', how='left')
print('After the scale:', book[['score', 'bucket', 'grade']].isna().sum().to_dict())
print(book['score'].count(), 'of', len(book), '| mean:', round(book['score'].mean(), 2))
unrated = book[book['rating'].isna()]
unrated[['ticker', 'sector', 'oas', 'market_value']]
Gaps: {'rating': 5}
After the scale: {'score': 5, 'bucket': 5, 'grade': 5}
195 of 200 | mean: 9.35
```

The merge turned one gap into four, and the mean score of 9.35 is over 195 bonds: "mean()" skips a gap, as pandas Part 1's "sum()" did.

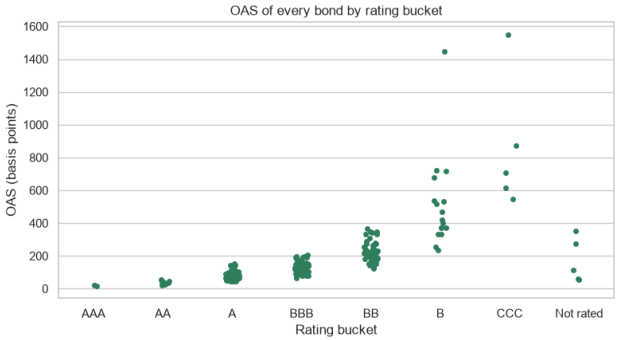
Decision 4: Fill With a Label

```
book['bucket'] = book['bucket'].fillna('Not rated')
book['grade'] = book['grade'].fillna('Not rated')
bucket_order = bucket_order + ['Not rated']
```

Try each treatment on a copy, print what it does, and write the choice down. A column more than half empty is usually dropped whole, with "drop(columns=)".

	ticker	sector	oas	market_value
26	CORL	Materials	56	4650820.0
70	GALX	Materials	350	4881337.5
105	MORX	Telecom	273	1529605.0
109	NEVE	Industrials	113	807900.0
150	SYLX	Transportation	59	3472875.0

Five bonds in four sectors, OAS from 56 to 350 bp: no cluster, which would itself be a finding.



At 56 bp one sits with AA and A, at 350 bp another with BB and B: the case against one guess for all five.

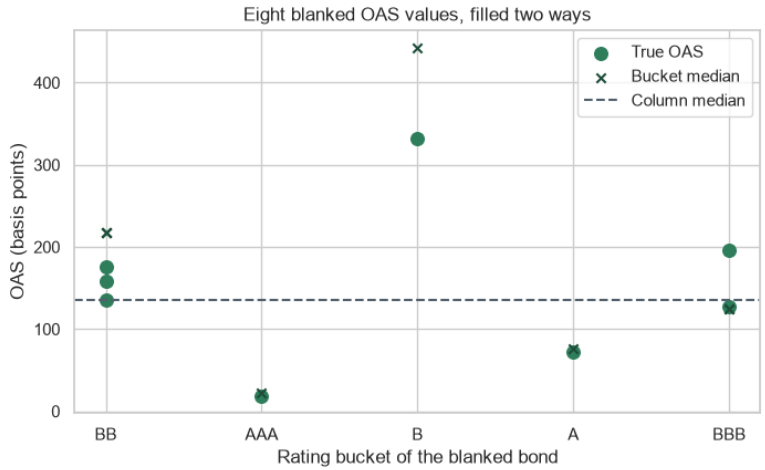
Fill a number from its group, and test the fill first

```
test = book.copy()
blanked = test.sample(8, random_state=1).index
test.loc[blanked, 'oas'] = np.nan
buckets = test.loc[blanked, 'bucket']
print('Blanked:', test['oas'].isna().sum(), '| their buckets:', buckets.tolist())
Blanked: 8 | their buckets: ['BB', 'AAA', 'BB', 'BB', 'B', 'A', 'BBB', 'BBB']
```

```
column_fill = test['oas'].fillna(test['oas'].median())
bucket_fill = test['oas'].fillna(test.groupby('bucket')['oas'].transform('median'))
miss_by_column = (column_fill - book['oas'])[blanked].abs().mean()
miss_by_bucket = (bucket_fill - book['oas'])[blanked].abs().mean()
print('Average miss, column median:', round(miss_by_column, 1), 'bp')
print('Average miss, bucket median:', round(miss_by_bucket, 1), 'bp')
Average miss, column median: 63.6 bp
Average miss, bucket median: 47.1 bp
```

Excel side by side. Rating in F, market value in S

Job	pandas	Excel
Count gaps	`isna().sum()`	=COUNTBLANK(F2:F201)
Value with no rating	`.sum() on the five`	=SUMIF(F2:F201, "", S2:S201)
Skip gaps	`mean()`	AVERAGE skips blanks;
		=AGGREGATE(1, 6, ...) skips #N/A too
Fill every gap	`fillna()`	Go To Special, Blanks, type, Ctrl+Enter

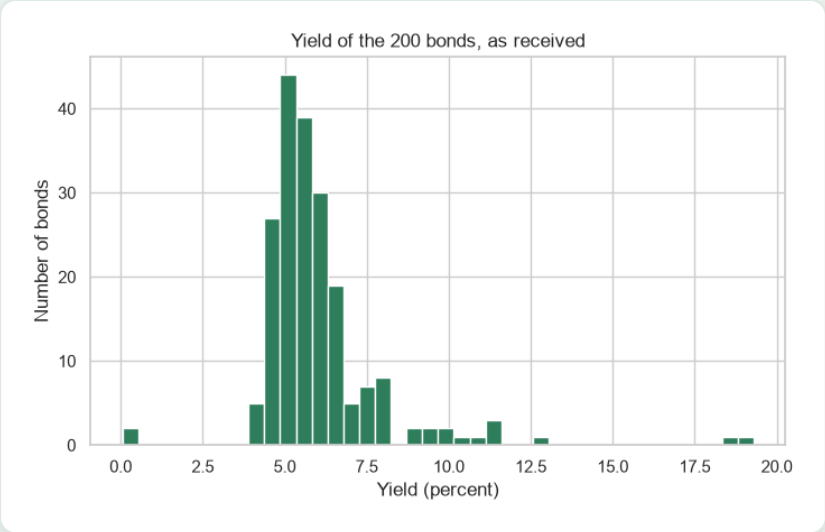


The column median hands all eight the same 135; the bucket median follows the rating.

```
average_miss('oas')
oas | column median: 102.57 | bucket median: 55.37
```

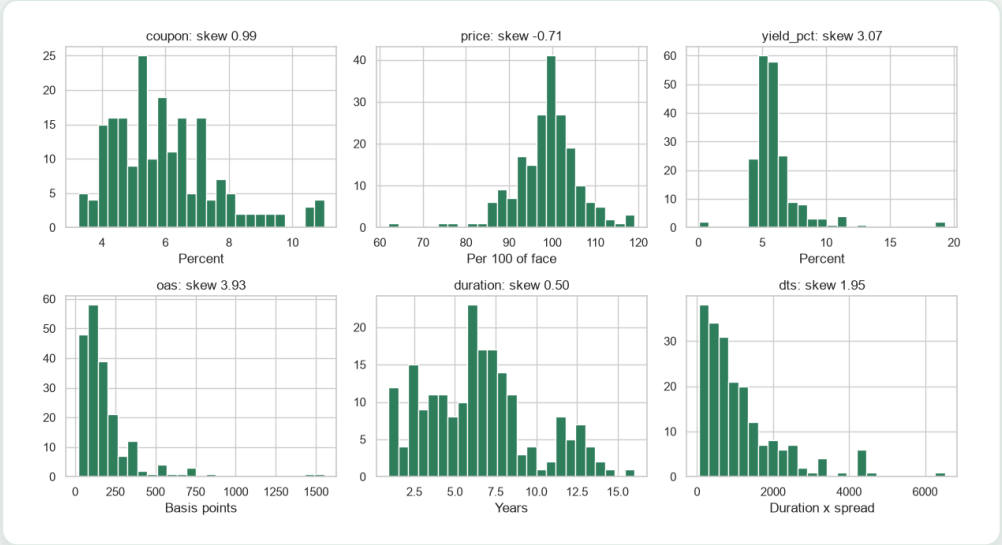
On OAS the bucket median misses by about half as much: a rating says a lot about a spread.

13.4 One column at a time: shape, centre, and skew




```
print(book['yield_pct'].nsmallest(4).tolist())
[0.04706, 0.07308, 4.089, 4.127]
```

Two bars stand alone near zero, four points below the next yield: the chart shows how many and how far.




```
columns = ['price', 'yield_pct', 'oas', 'duration', 'dts']
book[columns].agg(['mean', 'median', 'skew']).round(2)
```

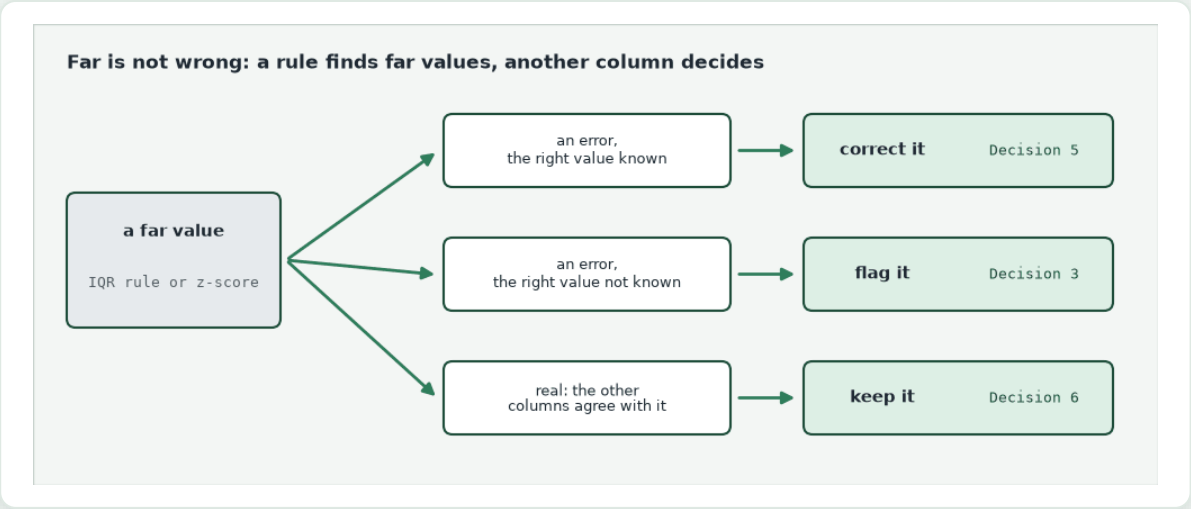
	price	yield_pct	oas	duration	dts
mean	98.83	6.03	185.75	6.47	1133.22
median	99.42	5.53	135.50	6.31	811.00
skew	-0.71	3.07	3.93	0.50	1.95

Skew measures how lopsided a column is: positive with a tail on the right, negative on the left, near 0 when the halves match.

OAS skews 3.93, its mean 50 bp above its median; price skews left, as Visualization drew it.

One minimum looks wrong and a few bonds stand far out. Next: error, or real.

13.5 Outliers: far is not wrong



Far has two causes, an error or a real extreme, and a rule cannot tell them apart.

```
q1, q3 = book['yield_pct'].quantile(0.25), book['yield_pct'].quantile(0.75)
lower_limit, upper_limit = q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1)
low_yield = book['yield_pct'] < lower_limit
high_yield = book['yield_pct'] > upper_limit
inside = ~low_yield & ~high_yield
print('Below:', low_yield.sum(), '| above:', high_yield.sum(), '| inside:', inside.sum())
Below: 2 | above: 14 | inside: 184
```

	ticker	rating	coupon	price	yield_pct	oas
123	PLEA	AA	7.25	119.131	0.04706	38
152	THAA	BB	7.00	96.623	0.07308	275

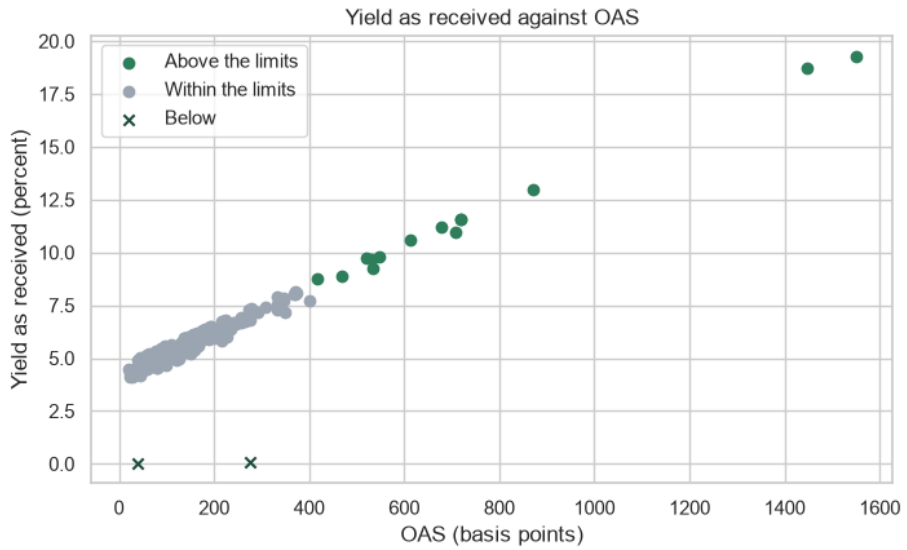
A bond below par yields more than its coupon: THAA at 96.623 should yield over 7 percent, not 0.073.

```
base = book['yield_pct'] - book['oas'] / 100
print('The two:', base[low_yield].round(3).tolist())
others = base[~low_yield]
print('The other 198:', round(others.min(), 3), 'to', round(others.max(), 3))
times_100 = book.loc[low_yield, 'yield_pct'] * 100 - book.loc[low_yield, 'oas'] / 100
print('The two, times 100:', times_100.round(3).tolist())
The two: [-0.333, -2.677]
The other 198: 3.691 to 4.585
The two, times 100: [4.326, 4.558]
```

Times 100, both land inside the other 198's range: entered as decimals, with the right value known.

Sixteen bonds past the limits: Visualization's box would draw them as sixteen circles, all alike.

The fourteen highest: real high yield bonds, kept



The fourteen sit on the line the other 184 draw; the two crosses sit far off it.

An error breaks a relationship; a real extreme keeps it.

```
z = (book['oas'] - book['oas'].mean()) / book['oas'].std()
print('Std:', round(book['oas'].std(), 2), '| top z:', round(z.max(), 2))
print('Above 3:', (z > 3).sum(), '| above 2:', (z > 2).sum())
print('OAS above 3:', book.loc[z > 3, 'oas'].tolist())
wide = book['oas'] > 409.875
print('Past 409.875:', wide.sum(), '| same as the high yields:', (wide == high_yield).all())
print('Their ratings:', sorted(book.loc[wide, 'rating'].unique()))
Std: 193.95 | top z: 7.04
Above 3: 3 | above 2: 8
OAS above 3: [1446, 1551, 871]
Past 409.875: 14 | same as the high yields: True
Their ratings: ['B', 'B+', 'B-', 'CCC', 'CCC+']
```

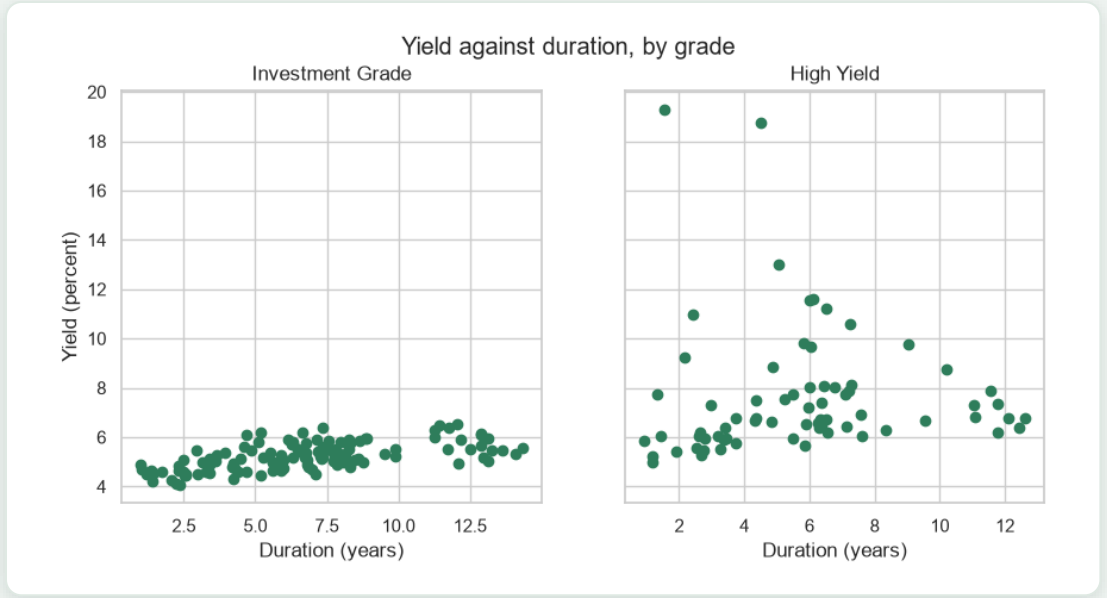
The fourteen high yields are the book's fourteen widest spreads, all rated B+ or lower: real high yield bonds.

Excel side by side. Yield in L, OAS in M

Job	pandas	Excel
Skew	`skew()`	=SKEW(M2:M201)
Quartiles	`quantile(0.25)`	=QUARTILE.INC(L2:L201, 1)
z-score	`(x - mean) / std`	=STANDARDIZE(M2, AVERAGE(M$2:M$201), STDEV.S(M$2:M$201))
Cap	`clip(upper=409.875)`	=MIN(M2, 409.875)

13.6 Two columns at a time: one number, two grades

```
print('Yield and OAS, as received:', round(received_yield.corr(book['oas']), 2))
print('Yield and OAS, corrected:', round(book['yield_pct'].corr(book['oas']), 2))
print('Duration and yield, book:', round(book['duration'].corr(book['yield_pct']), 2))
for grade in ['Investment Grade', 'High Yield']:
    part = book[book['grade'] == grade]
    print(grade, len(part), 'bonds:', round(part['duration'].corr(part['yield_pct']), 2))
Yield and OAS, as received: 0.95
Yield and OAS, corrected: 0.99
Duration and yield, book: 0.01
Investment Grade 125 bonds: 0.61
High Yield 70 bonds: 0.0
```



Two errors in 200 moved a correlation built from means; Visualization's 0.99 holds on the corrected column.

Investment grade yields rise with duration; high yield is set by credit, and the book's 0.01 describes neither grade.

Excel side by side. Duration in N, yield in L, grade in V

Job	pandas	Excel
Correlation	`corr()`	=CORREL(L2:L201, M2:M201)
Within one grade	`a filter, then corr()`	=CORREL(FILTER(N2:N201, V2:V201="High Yield"), FILTER(L2:L201, V2:V201="High Yield")), Excel 2021 and 365
Leave out the circles	`showfliers=False`	Box and Whisker (Excel 2016 and later), Format Data Series, untick Show outlier points

"CORREL" on a filtered sheet still counts the hidden rows.

Predict first: grades summed, fill on duration

Predict 1

The next cell adds up market value by grade, then adds the grades together. What comes back? A) 490,819,215, the book. B) Less than the book. C) An error: five grades are missing.

```
by_grade = book.groupby('grade')['market_value'].sum()
print(by_grade.to_dict())
print('Sum of the grades:', by_grade.sum(), '| book:', book['market_value'].sum())
```

	ticker	sector	oas	market_value
26	CORL	Materials	56	4650820.0
70	GALX	Materials	350	4881337.5
105	MORX	Telecom	273	1529605.0
109	NEVE	Industrials	113	807900.0
150	SYLX	Transportation	59	3472875.0

Five bonds in four sectors, OAS from 56 to 350 bp: no cluster, which would itself be a finding.

Predict 2

Over the same 100 draws, how does the bucket median do on duration? A) Clearly better than the column median. B) About level with it. C) Clearly worse.

```
average_miss('duration')
```

```
average_miss('oas')
oas | column median: 102.57 | bucket median: 55.37
```

On OAS the bucket median misses by about half as much: a rating says a lot about a spread.

Predict first: times 100, and OAS without the 14

Predict 3

The next cell multiplies the two yields by 100. What happens to the mean and the median yield? A) Both rise by about the same. B) The mean rises about 0.06, the median much less. C) The mean rises about 0.6, the median not at all.

```
received_yield = book['yield_pct'].copy()
book.loc[low_yield, 'yield_pct'] = (book.loc[low_yield, 'yield_pct'] * 100).round(3)
headline('Decision 5', book)
print('Mean moved:', round(book['yield_pct'].mean() - received_yield.mean(), 4),
      '| median moved:', round(book['yield_pct'].median() - received_yield.median(), 4))
```

Predict 4

The mean OAS is 185.75 bp. Take out the 14 bonds past 409.875 bp: what is the mean OAS of the rest? Write down a number.

```
trimmed = book[~wide]
headline('Without the 14', trimmed)
print('Median OAS:', trimmed['oas'].median())
print('Market value removed:', book.loc[wide, 'market_value'].sum())
capped = book['oas'].clip(upper=409.875)
print('Capped:', (capped != book['oas']).sum(), '| mean:', round(capped.mean(), 2),
      '| median:', capped.median())
```

13.7 and 13.8 Next month in one line

```
rebuilt = clean_holdings(raw, ratings)
print(rebuilt.shape, "| same as today's book:", rebuilt.equals(book), '| raw:', raw.shape)
(200, 22) | same as today's book: True | raw: (204, 19)
```

```
assert len(raw) == expected_bonds, f'{len(raw)} rows; the note says {expected_bonds}'
AssertionError: 204 rows; the note says 200
```

Print for the reader; assert for the run nobody watches.

	problem	rows	decision
0	Rows sent twice	4	removed
1	Untidy tickers	8	tidied
2	Stale prices	6	flagged for ops
3	No rating	5	labelled Not rated
4	Yields as decimals	2	times 100
5	Very wide OAS	14	kept

Three investment grade, two high yield, not one BBB-: the guess would have been wrong five times out of five.

The note asked	Answer
Can the file be trusted?	After six decisions, it matches the clean file
The list for ops	4 copies, 8 tickers, 6 August prices, 5 ratings, 2 yields
Next month	one line, clean_holdings(raw, ratings), checked by assert

Pack, list for ops, workbook, and next month in one line: the note is answered.

Next, the Treasury yields case study puts the same order of work on a new kind of table: one row a day for over eleven years.

Excel side by side. The dates open as "########" until the column is widened.

Job	pandas	Excel
The same steps next month	`clean_holdings(raw, ratings)`	Power Query: Applied Steps, then Refresh
A control check	`assert`	=IF(COUNTA(B2:B201)=200, "ok", "CHECK"), which shows and stops nothing